UNITÀ 6 · INTRODUZIONE AI SISTEMI OPERATIVI

Simulazione della verifica Unità 6

Stessa struttura e stessa difficoltà della verifica vera, con domande diverse. Fai la prova in 60 minuti senza guardare gli appunti, poi correggiti con le soluzioni in fondo.

Nome e cognome | Classe | Data

1. Scelta multipla

18 punti · 3 per domanda

Segna una sola risposta per ogni domanda.

a) Visto "dall'alto", da chi programma, il sistema operativo è…

- ○ una macchina estesa che offre astrazioni comode
- ○ un gestore delle risorse
- ○ un compilatore
- ○ un driver

b) Quale di questi kernel è ibrido?

- ○ Linux
- ○ Minix 3
- ○ QNX
- ○ Windows NT

c) A che cosa serve una chiamata di sistema?

- ○ A chiedere un servizio al kernel
- ○ A chiamare un altro utente
- ○ A riavviare il sistema
- ○ A installare un driver

d) Una videochiamata richiede un sistema…

- ○ hard real time
- ○ soft real time
- ○ batch
- ○ embedded

e) Quale kernel usa iOS?

- ○ Linux
- ○ Windows NT
- ○ XNU
- ○ Minix

f) In 5 minuti la CPU è occupata per 4 minuti. L'utilizzo è…

- ○ 125%
- ○ 80%
- ○ 20%
- ○ 4%

2. Vero o falso

12 punti · 1,5 per frase

V ☐ F ☐ Il kernel resta in memoria dall'accensione allo spegnimento.
V ☐ F ☐ Un errore in user mode ferma sempre tutto il sistema.
V ☐ F ☐ Un processo in esecuzione a cui scade il quanto passa nello stato di attesa.
V ☐ F ☐ Il PCB contiene il program counter del processo.
V ☐ F ☐ Con FCFS l'ordine di arrivo dei processi non cambia l'attesa media.
V ☐ F ☐ Se il quanto è più lungo di tutti i burst, il Round Robin si comporta come FCFS.
V ☐ F ☐ Gli interrupt hardware possono arrivare in qualunque momento.
V ☐ F ☐ Android usa il kernel Linux.

3. In laboratorio

6 punti · 1 per collegamento

1. `Get-Process` (PowerShell)	a. Mostra da quanto è acceso Linux e il carico medio
2. `free -h`	b. Termina il processo con quel PID
3. `uptime`	c. Elenca i processi di Windows come oggetti
4. `systeminfo`	d. Apre Gestione dispositivi, dove si vedono gli IRQ
5. `devmgmt.msc`	e. Mostra la RAM totale, usata e libera di Linux
6. `taskkill /pid 4812`	f. Mostra le caratteristiche di Windows e l'ora dell'ultimo avvio

Risposte: 1 ___ 2 ___ 3 ___ 4 ___ 5 ___ 6 ___

4. Collega

6 punti · 1 per collegamento

1. Batch	a. Un pacemaker
2. Time sharing	b. Il telefono che deve durare tutto il giorno con la batteria
3. Hard real time	c. Migliaia di richieste a un modello di AI elaborate in blocco entro alcune ore
4. Soft real time	d. La centralina della caldaia
5. Mobile	e. Lo streaming di una partita
6. Embedded	f. Il CTSS del MIT, 1961: tanti utenti ai terminali, a turno

Risposte: 1 ___ 2 ___ 3 ___ 4 ___ 5 ___ 6 ___

5. Completa

8 punti · 1 per spazio

1. Il programma che fa da interfaccia tra l'utente e il kernel è la _____.
2. Nei processori x86 le applicazioni lavorano nel ring _____.
3. Un processo che ha tutto tranne la CPU è nello stato _____.
4. Un "filo" di esecuzione che condivide la memoria del suo processo è un _____.
5. Il numero che identifica un processo è il _____.
6. Il comando Linux che elenca i processi è _____.
7. La procedura del kernel che gestisce un interrupt è la routine di _____.
8. Il dispositivo che interrompe la CPU a intervalli regolari è il _____.

6. Scheduling della CPU

20 punti · 6 + 6 + 6 + 2

Quattro processi arrivano tutti al tempo 0 e sono in coda nell'ordine **A, B, C, D**. Burst: **A = 4, B = 6, C = 1, D = 3**. Per ogni algoritmo scrivi nella griglia la lettera del processo che usa la CPU in ogni unità di tempo, poi completa la tabella (attesa = completamento − burst).

Tempo	0	1	2	3	4	5	6	7	8	9	10	11	12	13
a) FCFS														
b) SJF														
c) RR, q = 2														

	a) FCFS		b) SJF		c) Round Robin, q = 2	
Processo	**Completam.**	**Attesa**	**Completam.**	**Attesa**	**Completam.**	**Attesa**
A						
B						
C						
D						
Media						

d) Quale algoritmo dà l'attesa media più bassa? Con quale tutti i processi usano la CPU per la prima volta entro 5 unità di tempo?

7. Gli stati di un processo

8 punti · 2 per evento

Per ogni evento scrivi lo stato di partenza e lo stato di arrivo del processo (nuovo, pronto, esecuzione, attesa, terminato).

Evento	Da	A
a) Il sistema ha creato il PCB e caricato il programma in memoria		
b) Il processo aspetta che l'utente prema un tasto		
c) Il processo esegue la sua ultima istruzione e chiude		
d) Lo scheduler ridà la CPU a un processo in coda		

8. Misurare le prestazioni

8 punti · 4 + 4

a) In 3 minuti la CPU completa 90 processi ed è occupata per 150 secondi. Calcola il throughput (al secondo e al minuto) e l'utilizzo della CPU.

b) Un job usa la CPU per 30 ms, poi aspetta l'I/O per 90 ms, e ripete. Qual è l'utilizzo della CPU se in memoria c'è solo lui? Che cosa fa un sistema multiprogrammato durante i 90 ms?

9. Gli interrupt

8 punti · 4 + 2 + 2

a) Numera da 1 a 6 i passi della gestione di un interrupt della scheda di rete.

____	La CPU salva program counter e registri e passa in kernel mode
____	La routine di servizio copia il pacchetto arrivato in memoria
____	La CPU termina l'istruzione che stava eseguendo
____	La scheda di rete invia una richiesta di interrupt (IRQ)
____	Lo stato viene ripristinato e il processo interrotto riprende
____	La CPU cerca nel vettore delle interruzioni l'indirizzo della routine

b) La CPU controlla la tastiera con il polling 2000 volte al secondo e tu premi 4 tasti al secondo. Quanti controlli al secondo sono inutili? Che percentuale?

c) Un timer genera interrupt alla frequenza di 100 Hz. Ogni quanti millisecondi arriva un interrupt?

10. Domanda aperta

6 punti

Spiega la differenza tra polling e interrupt e perché, senza l'interrupt del timer, il sistema operativo non potrebbe realizzare il Round Robin. Usa almeno quattro di questi termini: polling, interrupt, timer, quanto, scheduler, kernel mode, prelazione.

Punti: _____ / 100 **Voto: _____**

Soluzioni

Correggi la tua prova, somma i punti e dividi per 10: ottieni il voto in decimi. Per ogni errore torna alla lezione su quell'argomento e rifai l'esercizio.

1. Scelta multipla · 3 punti ciascuna

a) **una macchina estesa che offre astrazioni comode** · b) **Windows NT** (Linux è monolitico, Minix 3 e QNX sono microkernel) · c) **a chiedere un servizio al kernel** · d) **soft real time** (un ritardo peggiora la qualità, ma non è un guasto) · e) **XNU**, lo stesso di macOS · f) **80%** (240 s ÷ 300 s)

2. Vero o falso · 1,5 punti ciascuna

1 **V** · 2 **F** (il kernel termina solo quel processo) · 3 **F** (torna pronto: non aspetta un evento, solo il suo turno) · 4 **V** · 5 **F** (effetto convoglio) · 6 **V** · 7 **V** (sono asincroni) · 8 **V**

3. In laboratorio · 1 punto ciascuno

1c · 2e · 3a · 4f · 5d · 6b

4. Collega · 1 punto ciascuno

1c · 2f · 3a · 4e · 5b · 6d

5. Completa · 1 punto ciascuno

shell · **3** · **pronto** · **thread** · **PID** · **ps** (accettato top) · **servizio** (ISR) · **timer**

6. Scheduling della CPU · 20 punti

Algoritmo	Griglia (tempi da 0 a 13)	Completamento A, B, C, D · media	Attesa A, B, C, D · media
a) FCFS	`AAAABBBBBBCDDD`	4, 10, 11, 14 · **9,75**	0, 4, 10, 11 · **6,25**
b) SJF	`CDDDAAAABBBBBB`	8, 14, 1, 4 · **6,75**	4, 8, 0, 1 · **3,25**
c) RR, q = 2	`AABBCDDAABBDBB`	9, 14, 5, 12 · **10**	5, 8, 4, 9 · **6,5**

RR passo per passo: A 0–2 (restano 2), B 2–4 (restano 4), C 4–5 (termina), D 5–7 (resta 1), A 7–9 (termina), B 9–11 (restano 2), D 11–12 (termina), B 12–14 (termina). d) **SJF** ha l'attesa media più bassa; con il **Round Robin** tutti partono entro 5 unità (A 0, B 2, C 4, D 5). Per ogni algoritmo: Gantt 2 punti, completamenti 2, attese e medie 2.

7. Gli stati di un processo · 2 punti ciascuno

a) **nuovo → pronto** · b) **esecuzione → attesa** · c) **esecuzione → terminato** · d) **pronto → esecuzione**

8. Misurare le prestazioni · 8 punti

a) 3 minuti = 180 s. Throughput = 90 ÷ 180 = **0,5 processi al secondo = 30 al minuto**; utilizzo = 150 ÷ 180 = **83,3%**. b) 30 ÷ (30 + 90) = **25%**; durante l'attesa il sistema multiprogrammato **dà la CPU a un altro job** già in memoria.

9. Gli interrupt · 8 punti

a) Numeri nell'ordine delle righe: **3, 5, 2, 1, 6, 4** (IRQ, fine istruzione, salvataggio, vettore, routine, ripristino). b) 2000 − 4 = **1996**, cioè il **99,8%**. c) 1 ÷ 100 = 0,01 s = **10 ms**.

10. Domanda aperta · 6 punti

Esempio di risposta: con il **polling** è la CPU a controllare di continuo se un dispositivo ha dati, sprecando tempo quando gli eventi sono rari; con l'**interrupt** è il dispositivo a mandare un segnale e la CPU lavora ad altro finché non arriva. Nel Round Robin ogni processo può usare la CPU al massimo per un **quanto**; ma un processo che calcola senza mai chiedere I/O non restituirebbe mai la CPU da solo. Il **timer** genera un interrupt a intervalli regolari: la CPU passa in **kernel mode**, lo **scheduler** controlla se il quanto è scaduto e, se sì, toglie la CPU al processo (**prelazione**) e la dà al successivo. Punti: polling e interrupt 2, ruolo del timer nel Round Robin 2, termini usati correttamente 2.